

5.10 Usporedba nestrukturnog, strukturnog i objektnog pristupa programiranju

5.10 Usporedba nestrukturnog, strukturnog i objektnog pristupa programiranju

	nestrukturni pristup	strukturni pristup	objektni pristup
kontrolne strukture	sekvenca, selekcija, iteracija, skokovi	sekvenca, selekcija, iteracija	sekvenca, selekcija, iteracija
složeni tipovi podataka	matrice	- matrice, strukture, skupovi	matrice, strukture, skupovi
lokalni podaci	ne	- podaci lokalni za potprograme - podaci lokalni strukturama	- podaci lokalni za potprograme - podaci lokalni strukturama - podaci lokalni objektima
lokalni potprogrami	ne	- potprogrami lokalni drugim potprogramima	- potprogrami lokalni drugim potprogramima - potprogrami lokalni objektima
skrivanje podataka	ne	podaci lokalni potprogramima	- podaci lokalni potprogramima, - privatni i zaštićeni članovi klasa - privatno i zaštićeno nasljeđivanje
nasljeđivanje	ne	ne	nasljeđivanje klasa objekata
polimorfizam	ne	primitivni polimorfizam podataka (istoimeni globalni i lokalni podaci)	- istoimene metode s različitom implementacijom unutar iste hijerarhije klasa
enkapsulacija (spajanje podataka i operacija)	ne	na razini potprograma	- na razini potprograma - na razini objekta

Prema Motiku i Šribaru ("Demistificirani C++", 1997., str. 3) proceduralno strukturirano programiranje zasniva se na promatranju programa kao niza jednostavnih programskih odsječaka, potprograma. Svaki potprogram predstavlja program u malom : ima svoj naziv, ulazne i izlazne vrijednosti, te svoje lokalne podatke. Zamišljen je tako da predstavlja logičku cjelinu unutar programa, što olakšava kasnije ispravljanje eventualnih pogrešaka, te pozivanje iz različitih dijelova programa, tako da isti programski kod nije potrebno pisati na više mjesta u programu. Odnosno, cijeli program je presložen da bi se mogao razumjeti, pa ga se zato dijeli na više manjih dijelova – potprograma, koji se mogu realizirati pomoću naredbi odgovarajućeg programskog jezika. U odnosu na nestrukturirano programiranje, postiglo se to da program ima jasnu hijerarhijsku strukturu (što se postiglo izbacivanjem skokova, odnosno naredbe *goto*), što su logičke cjeline unutar programa izdvojene u potprograme i na kraju, striktno odvajanje podataka od njihove obrade.

Osnovni problem kod takvog, strukturnog pristupa programiranju je upravo ovo zadnje, odnosno odvajanje podataka i potprograma za njihovu obradu. Naime, svi potprogrami načelno su dostupni iz svih dijelova programa, što znači da moraju imati različite nazive. To dovodi do problema kada želimo unutar iste podatkovne strukture voditi različite podatke, jer

je potrebno najprije ispitati tip svakog pojedinog podatka, te prema njemu pozvati odgovarajući potprogram.

Primjer:

Treba napisati program za evidenciju knjiga. Knjige koje treba evidentirati mogu biti udžbenici, rječnici ili romani. Zajednički podaci sve tri vrste knjiga su godina izdanja, autor(i), izdavač i broj stranica ali također postoje i podaci koji se vode samo za pojedine vrste knjiga:

- za udžbenike je potrebno voditi i kolegij na koji se odnose,
- za rječnike treba voditi podatke o jezicima koje pokrivaju i broju riječi.

Ako bi koristili strukturni pristup, odgovarajuću strukturu podataka mogli bi definirati na slijedeći način (u Pascalu) :

```
type
tknjiga = record
    vrsta_knjige : byte;
    godina_izd : integer;
    autor : string [50];
    izdavac : string [20];
    broj_str : integer;
    kolegij : string [20];
    jezici : string [30];
end;
```

Dakle, morali smo definirati takvu strukturu podataka koja obuhvaća sve moguće podatke koji bi se mogli pojaviti za neku knjigu, što dovodi do redundancije, odnosno nepotrebnog utroška memorije. Također, moramo voditi i podatak o vrsti knjige, kako bi mogli znati koje podatke ćemo zatražiti za upis, odnosno ispisivati ih kad bude potrebno.

Nadalje, kod unosa podataka morat ćemo koristiti selekciju da bi odredili odgovarajući potprogram za unos:

```
write ('Unesite vrstu knjige (1.udžbenik, 2.rječnik, 3.roman) : ');
readln (knjiga.vrsta_knjige);
case knjiga.vrsta_knjige of
    1: unos_udz;
    2: unos_rjec;
    3: unos_rom;
end;
```

Istu selekciju za izbor potprograma trebat ćemo i za ispis podataka o nekoj knjizi:

```
case knjiga.vrsta_knjige of
    1: ispis_udz;
    2: ispis_rjec;
    3: ispis_rom;
end;
```

Objektni pristup programiranju uvodi pojmove klase i objekta. Klasa udružuje podatke i potprograme za obradu tih podataka – metode. To udruživanje podataka i operacija nad tim podacima naziva se *enkapsulacija*. Cijeli program podijeljen je na više zatvorenih cjelina, objekata, koji međusobno surađuju u rješavanju problema. Nisu svi podaci ni sve operacije jednog objekta vidljivi izvan tog objekta, odnosno, postoji vanjsko sučelje objekta, dok detalji implementacije nisu dostupni izvan objekta. Slijedeće što uvodi objektno orijentirano

programiranje je nasljeđivanje. Nasljeđivanjem izvedena klasa preuzima podatke i metode osnovne klase, čime se osigurava ponovna iskoristivost programskog koda, ali izvedena klasa može imati i svoje vlastite podatke i metode. Naslijeđene metode nasljeđuju implementaciju iz osnovne klase, ali mogu imati i novu, vlastitu implementaciju. U tom slučaju postiže se slijedeće važno svojstvo objektno orijentiranog programiranja : *polimorfizam*. Polimorfizam omogućuje različite implementacije istoimenih metoda, a budući da su te metode pridružene podacima unutar odgovarajućih objekata, to mogu biti i njima prilagođene.

Zadatak iz prethodnog primjera mogao bi se korištenjem objektnog pristupa riješiti na slijedeći način (u C++):

1. Definiramo osnovnu apstraktnu klasu *tknjiga*, koja sadrži zajedničke elemente svih knjiga:

```
class tknjiga{
public:
    int godina_izd;
    char autor [50];
    char izdavac [20];
    int broj_str;
    virtual void unos()=0;
};
```

2. Definiramo izvedene klase :

```
class tudzbenik : public tknjiga{
public:
    char kolegij [20];
    void unos(){ // metoda za unos podataka o udžbeniku
        .
        .
    };
};
class trjecnik : public tknjiga{
public:
    char jezici [30];
    void unos(){ // metoda za unos podataka o rječniku
        .
        .
    };
};
class troman : public tknjiga{
public:
    void unos(){ // metoda za unos podataka o romanu
        .
        .
    };
};
```

3. Definiramo pokazivač na objekt osnovne klase :

```
tknjiga *knjiga;
```

4. Ovisno o vrsti knjige, alociramo objekt odgovarajuće klase:

```
cout << "Unesite vrstu knjige (1.udžbenik, 2.rječnik, 3.roman) : ";
cin >> vrsta;
switch (vrsta){
```

```
case 1: knjiga=new tudzbenik;  
case 2 : knjiga=new trjecnik;  
case 3 : knjiga=new troman;  
};
```

Ovdje vidimo da objekt *knjiga* istovremeno pripada osnovnoj klasi i jednoj od izvedenih klasa.

5. Kada je objekt alociran, možemo pozivati njegove metode, bez obzira na to kojoj od izvedenih klasa pripada:

```
knjiga->unos();
```

Poziva se metoda *unos*, koja se razlikuje ovisno o tome radi li se o udžbeniku, rječniku ili romanu.

Ono što je bitno za odnos strukturnog i objektnog pristupa programiranju, je to da objektno orijentirani pristup programiranju sadrži u sebi strukturni pristup, odnosno, predstavlja njegovu nadgradnju. Snaga objektno orijentiranog pristupa leži ponajprije u polimorfizmu te mogućnosti da objekti izvedenih klasa istovremeno pripadaju i osnovnoj klasi. Time se postiže da se unutar iste podatkovne strukture (to je kod objektno orijentiranog programiranja tipično vezana lista) vode međusobno heterogeni elementi, a da nije potrebno pisati poseban programski kod da bi se ispitali njihovi tipovi i pozvali odgovarajući potprogrami. Umjesto toga, postoji mehanizam poziva virtualnih metoda, koji vodi računa o tome koja metoda treba biti pozvana u odgovarajućem slučaju.

Ipak, objektno orijentirani pristup programiranju ima i svoje nedostatke, od kojih je najvažniji krutost objektno strukture. Pojedinu klasu u sustavu nasljeđivanja nije moguće "istrgnuti", odnosno razumjeti je bez poznavanja njenih nadređenih i podređenih klasa. Ukoliko je hijerarhija klasa u nekom svom dijelu pogrešno postavljena, teško je pogrešku ispraviti. Zbog toga strukturni pristup i dalje ostaje aktualan, odnosno, često se ta dva pristupa kombiniraju. Takav je i jezik C++, koji u sebi uključuje kompletan jezik C, koji je strukturno orijentiran, ali omogućuje i objektno orijentirano programiranje.